Trabajo Agente

Desarrollo de un agente inteligente con LangChain, LangGraph y RAG

1. Presentación del trabajo

1.1. Descripción general

En este trabajo deben diseñar, implementar y desplegar una aplicación web completa cuyo núcleo sea un agente inteligente construido con LangChain y LangGraph. El agente responderá preguntas sobre un dominio específico a partir de una base de conocimiento propia, almacenada en una base de datos vectorial.

La solución integra, en un mismo producto, los temas trabajados durante el curso: diseño de una API, construcción de una interfaz de usuario, autenticación, orquestación de un agente mediante grafos, ejecución de modelos de lenguaje locales y despliegue en la nube. El proyecto abarca, por tanto, los siguientes componentes:

- Backend con API protegida.
- Frontend funcional.
- Autenticación de usuarios.
- Agente construido con LangChain y LangGraph.
- Modelo de lenguaje local ejecutado con Hugging Face u Ollama.
- Base de datos vectorial.
- Carga de archivos para ampliar la base de conocimiento.
- Despliegue del backend y del frontend en la nube.

1.2. Objetivo

Desarrollar una aplicación web que permita a un usuario autenticado interactuar con un agente inteligente capaz de responder preguntas fundamentadas en documentos y datos previamente cargados.

El agente debe construirse sobre una arquitectura **RAG (Retrieval Augmented Generation)**, de modo que cada respuesta se sustente en información efectivamente recuperada de la base de datos vectorial y no en el conocimiento paramétrico del modelo.

1.3. Modalidad de trabajo

El trabajo puede realizarse de manera individual o en parejas. En adelante, el documento se refiere a cualquiera de las dos modalidades como “el equipo”.

1.4. Temática del agente

El tema es libre, pero debe contar con información suficiente y accesible para construir una base de conocimiento útil. A modo de referencia, se sugieren los siguientes dominios:

- Harry Potter, La Casa de Papel, cine o anime.
- Mundiales de fútbol, Fórmula 1 u otros deportes.
- Videojuegos, música o libros.
- Historia de Colombia, universidades, turismo o gastronomía.

Antes de avanzar con la implementación, verifiquen que el tema escogido permite formular preguntas variadas y que existe material suficiente para alimentar la base vectorial.

2. Requisitos funcionales

2.1. Autenticación y gestión de usuarios

La aplicación debe permitir que los usuarios:

- Se registren o inicien sesión.
- Accedan al frontend únicamente si están autenticados.
- Consuman la API únicamente si cuentan con autorización.
- Mantengan una sesión activa.
- Cierren la sesión.

Se recomienda **Keycloak** para la gestión de autenticación y autorización, aunque pueden emplear otra solución o implementarla por su cuenta. En cualquier caso, la API debe estar protegida: ningún endpoint puede quedar expuesto al consumo sin autenticación.

2.2. Backend

El backend debe exponer una API REST o GraphQL que permita:

- Autenticar y validar usuarios.
- Enviar preguntas al agente y devolver sus respuestas.
- Subir archivos para complementar la base de conocimiento.
- Procesar los documentos cargados y guardar sus embeddings en la base vectorial.
- Consultar el historial de conversaciones del usuario.

- Listar los documentos cargados y eliminarlos, si aplica.

Tecnologías sugeridas: FastAPI, NestJS, Flask, Django REST Framework o Express.js.

El código debe estar bien estructurado, con una separación clara entre controladores, servicios, configuración, lógica del agente y acceso a datos.

2.3. Frontend

El frontend debe permitir:

- Registrarse e iniciar sesión.
- Interactuar con el agente mediante una interfaz tipo chatbot.
- Enviar preguntas y visualizar las respuestas.
- Subir archivos para alimentar la base de conocimiento.
- Consultar los documentos cargados.
- Mostrar mensajes de error cuando el agente no pueda responder.
- Cerrar sesión.

Tecnologías sugeridas: React, Next.js, Vue, Angular, Quasar o Svelte.

La interfaz debe ser clara, funcional y agradable. Pueden apoyarse en herramientas de inteligencia artificial para generarla.

2.4. Carga y procesamiento de documentos

La aplicación debe permitir subir nuevos archivos para ampliar la base de conocimiento del agente. Formatos sugeridos: PDF, TXT, Markdown, CSV, DOCX y archivos multimedia (JPG, JPEG, MP4, MP3).

El flujo de procesamiento debe cubrir los siguientes pasos:

1. Recibir el archivo desde el frontend y enviarlo al backend.
2. Extraer el contenido del documento.
3. Dividir el texto en fragmentos.
4. Generar los embeddings correspondientes.
5. Almacenar los embeddings en la base vectorial.
6. Confirmar al usuario que el documento fue procesado.

3. Requisitos técnicos

3.1. Agente con LangChain y LangGraph

El agente debe construirse obligatoriamente con **LangChain** y **LangGraph**, sobre una arquitectura basada en grafo. Un flujo de referencia es el siguiente:

1. Recibir la pregunta del usuario.
2. Validar la intención de la pregunta.
3. Buscar información relevante en la base vectorial.
4. Evaluar si la información recuperada es suficiente.
5. Responder con base en la información encontrada.
6. Rechazar la pregunta o pedir aclaración cuando el contexto sea insuficiente.
7. Guardar la interacción en el historial.

El agente no debe inventar información. Cuando no encuentre datos suficientes en la base de conocimiento, debe responder de manera explícita con un mensaje equivalente a:

> *No tengo suficiente información en mi base de conocimiento para responder esa pregunta.*

En esos casos también puede sugerir al usuario que cargue nuevos documentos relacionados con el tema.

3.2. Modelo local

El proyecto debe utilizar un modelo de lenguaje ejecutado localmente. No se permite depender únicamente de modelos comerciales en la nube. Pueden trabajar con:

- **Ollama**: Llama 3.1, Mistral, Gemma, Phi o Qwen.
- **Hugging Face**: transformers locales, Sentence Transformers para embeddings o modelos pequeños optimizados para ejecución local.

En el documento técnico deben justificar cuál modelo usaron, por qué lo escogieron, cómo se ejecuta localmente y qué limitaciones encontraron.

3.3. Base de datos vectorial

El sistema debe almacenar los embeddings de los documentos en una base de datos vectorial. Pueden emplear ChromaDB, FAISS, Qdrant, Weaviate, Milvus o PostgreSQL con pgvector.

La base vectorial debe permitir:

- Guardar los fragmentos de texto procesados.
- Asociar cada fragmento con su documento original.
- Buscar información relevante a partir de la pregunta del usuario.
- Recuperar el contexto con el que el agente construye la respuesta.

3.4. Control de alucinaciones

Este es uno de los criterios más importantes del trabajo: el agente no puede inventar respuestas. El sistema debe implementar al menos dos de las siguientes estrategias, o equivalentes:

- Umbral mínimo de similitud para aceptar los documentos recuperados.
- Verificación del contexto antes de responder.
- Prompt que obligue al agente a responder solo con la información recuperada.
- Nodo en LangGraph dedicado a validar si el contexto es suficiente.
- Respuestas explícitas de rechazo cuando no exista información.
- Presentación de las fuentes o los fragmentos usados para responder.

3.5. Despliegue en la nube

El backend y el frontend deben estar desplegados y ser accesibles mediante una URL pública.

- Frontend: Vercel, Netlify, GitHub Pages, Azure Static Web Apps o Firebase Hosting.
- Backend: Render, Railway, Azure App Service, AWS ECS, AWS EC2, Google Cloud Run, Fly.io o DigitalOcean.

En el documento técnico deben registrar la URL del frontend, la URL del backend, las variables de entorno utilizadas, el proceso de despliegue y los problemas que encontraron durante el mismo.

4. Restricciones

El flujo principal del agente debe construirse mediante código, con LangChain y LangGraph. Queda prohibido el uso de herramientas de tipo “drag and drop” o de automatización visual para construirlo, entre ellas n8n, Zapier, Make, Flowise, Botpress en modo visual y cualquier herramienta similar que genere el flujo del agente sin programación.

5. Entregables

Cada equipo debe entregar los cuatro productos que se resumen a continuación.

Entregable	Contenido	Formato
Repositorio en GitHub	Código, configuración, README y diagramas	Enlace público
Documento técnico	Los apartados descritos en la sección 5.2	PDF
Sustentación	Demostración en vivo de 7 a 10 minutos	Presencial
Diapositivas	Entre 8 y 12 láminas de apoyo	PDF

5.1. Repositorio en GitHub

El repositorio debe estar bien organizado, con commits significativos, y contener:

- Código del backend y del frontend.
- Configuración del agente y archivos de configuración del proyecto.
- README completo, con instrucciones para ejecutar localmente y para desplegar.
- Archivo de ejemplo con las variables de entorno.
- Capturas de pantalla y diagrama de arquitectura.

5.2. Documento técnico

El documento técnico debe desarrollar los siguientes apartados:

1. Nombre del proyecto e integrantes.
2. Tema seleccionado y descripción del problema.
3. Arquitectura general y diagrama de arquitectura.
4. Tecnologías utilizadas.
5. Descripción del backend.
6. Descripción del frontend.
7. Descripción del agente y del flujo de LangGraph.
8. Modelo local utilizado y base de datos vectorial.
9. Proceso de carga de documentos.
10. Estrategias implementadas para evitar alucinaciones.
11. Seguridad y autenticación.
12. Despliegue.
13. Evidencias de funcionamiento.

14. Problemas encontrados y conclusiones.

5.3. Sustentación

La sustentación tiene una duración de 7 a 10 minutos y debe incluir una demostración en vivo que cubra:

- Presentación del tema y de la arquitectura del sistema.
- Inicio de sesión del usuario y ejecución del frontend.
- Interacción con el agente, con preguntas correctamente respondidas.
- Preguntas que el agente rechaza por falta de información.
- Carga de un documento nuevo y una pregunta que dependa de ese documento.
- Revisión rápida del código y explicación del grafo construido con LangGraph.
- Evidencia del despliegue en la nube.

5.4. Diapositivas de apoyo

Las diapositivas acompañan la sustentación y deben tener entre 8 y 12 láminas, con la siguiente secuencia:

1. Nombre del proyecto e integrantes.
2. Problema o necesidad.
3. Arquitectura.
4. Tecnologías usadas.
5. Agente con LangChain y LangGraph.
6. Base vectorial y carga de documentos.
7. Seguridad y usuarios.
8. Demostración.
9. Conclusiones.

6. Evaluación

6.1. Requisitos mínimos para aprobar

Un proyecto que no cumpla con la totalidad de los siguientes puntos no obtiene nota aprobatoria, con independencia de la calidad del resto del trabajo:

- Backend y frontend funcionales.
- Autenticación implementada.
- Agente construido con LangChain y LangGraph.

- Modelo local en funcionamiento.
- Base vectorial integrada y carga de documentos operativa.
- Respuestas fundamentadas en el contexto recuperado.
- Rechazo explícito de las preguntas sin información suficiente.
- Despliegue en la nube y repositorio documentado.

6.2. Rúbrica

Criterio	Peso	Evidencia principal
Arquitectura general del sistema	10%	Documento técnico
Backend y API	15%	Repositorio
Frontend e interfaz de usuario	5%	Demostración
Autenticación y seguridad	10%	Demostración
Uso correcto de LangChain y LangGraph	15%	Repositorio
Modelo local (Ollama o Hugging Face)	8%	Documento técnico
Base de datos vectorial y RAG	10%	Repositorio
Carga y procesamiento de documentos	7%	Demostración
Control de alucinaciones	10%	Demostración
Despliegue en la nube	2%	URL pública
Documentación y sustentación	8%	Documento y diapositivas
Total	**100%**	

6.3. Criterios de calidad esperados

Además del cumplimiento funcional, se valorará que el proyecto presente:

- Código limpio y organizado, con separación clara entre frontend, backend y agente.
- Uso adecuado de variables de entorno.
- Manejo de errores y validación de formularios.
- Seguridad en los endpoints.
- README claro y commits frecuentes.
- Interfaz usable y respuestas coherentes del agente.
- Evidencia clara del uso de LangGraph y despliegue funcional.

7. Anexos

Anexo A. Estructura sugerida del repositorio

```
proyecto-agente/
│
├── backend/
│   ├── app/
│   ├── agents/
│   ├── services/
│   ├── controllers/
│   ├── vectorstore/
│   ├── requirements.txt
│   ├── Dockerfile
│   └── README.md
│
├── frontend/
│   ├── src/
│   ├── public/
│   ├── package.json
│   ├── Dockerfile
│   └── README.md
│
├── docs/
│   ├── arquitectura.png
│   ├── documento-tecnico.pdf
│   └── presentacion.pdf
│
├── docker-compose.yml
├── .env.example
└── README.md
```

Anexo B. Arquitectura de referencia

```
Usuario
   │
   ▼
Frontend Web
   │
   ▼
API Backend protegida
   ├── Autenticación (Keycloak u otro gestor)
   ├── Gestión de usuarios
   ├── Carga de archivos
   ├── Procesamiento de documentos
   └── Agente LangGraph
         ├── Nodo 1: recibir la pregunta
         ├── Nodo 2: buscar contexto en la base vectorial
         ├── Nodo 3: validar la relevancia del contexto
         ├── Nodo 4: generar la respuesta
         ├── Nodo 5: rechazar si no hay información suficiente
         └── Nodo 6: guardar el historial
               │
               ▼
         Modelo local (Ollama / Hugging Face)
               │
               ▼
         Base de datos vectorial
```

Anexo C. Ejemplos de preguntas esperadas

Suponiendo que el tema escogido sea Harry Potter, el agente debería responder correctamente preguntas como:

- ¿Quién es Albus Dumbledore?
- ¿Qué son los horrocruxes?
- ¿Cuál es la diferencia entre Gryffindor y Slytherin?
- ¿Qué pasó en la batalla de Hogwarts?
- ¿Qué información tienes sobre el Ministerio de Magia?

Frente a una pregunta ajena a la base de conocimiento, como “¿cuál es la mejor receta para preparar sushi?”, la respuesta esperada es un rechazo explícito:

> *No tengo suficiente información en mi base de conocimiento para responder esa pregunta.*

Anexo D. Stack recomendado para una implementación básica

- Backend: FastAPI.
- Frontend: React o Next.js.
- Autenticación: Keycloak, Supabaese.
- Modelo local: Ollama con Llama 3.1, Mistral o Gemma.
- Embeddings: Sentence Transformers o embeddings de Ollama.
- Base vectorial: ChromaDB o pgvector.
- Despliegue: Render, Railway o Azure App Service para el backend; Vercel o Netlify para el frontend.